

NeuralGraph: complete results of the retrieval research campaign

September 5 to 6, 2026. All runs on LoCoMo (10 conversations, 1,540 questions in four categories: single_hop 282, multi_hop 841, temporal 321, open_domain 96). Models: google/gemma-4-e4b for reranking, answering, and the campaign judge; nomic-embed-text v1.5 for embeddings; qwen/qwen3.6-35b-a3b as an independent judge. All numbers are leakage-free unless marked "old" (the December 2025 run used the gold answer as an acceptance gate and the gold category label for routing; both removed before any run below).

Two metrics are used throughout:

- **Accuracy**: an LLM judge labels the generated answer correct or wrong against the gold answer.
- **Recall@k**: any part of the gold answer (split on commas and "and", parts longer than two characters) appears as a substring in one of the top k retrieved messages. Retrieval-only, no LLM calls. Meaningless for the temporal category, whose gold answers are dates that live in metadata, so temporal is omitted from retrieval tables.

1. Headline: end-to-end accuracy of the shipped system

Shipped configuration: RETRIEVAL_MODE=local_pairs (per-agent memory routing, Tesseract top-50, 30 pair-node back-fills) plus open-domain prompt flags, 80-candidate pointwise rerank, 15 memories in the answer prompt (30 for list questions).

1.1 Conversations 1 to 5, 744 of 1,540 questions (run stopped early at the user's request)

Category	n	New system (Gemma, lenient)	Old leaky run, same questions (GPT-4o, lenient)
single_hop	142	73.9%	
multi_hop	400	72.8%	
temporal	156	73.7%	
open_domain	46	56.5%	
overall	744	72.2%	66.9%

The two columns use different judges, so the 5.3-point gap is indicative only.

1.2 Conversations 1 to 4, 584 questions, under four judges

Category	n	Old leaky (GPT-4o lenient)	Gemma lenient	Qwen lenient	Qwen strict	Substring
single_hop	111	47.7	75.7	62.2	33.3	21.6
multi_hop	311	75.2	72.3	66.6	41.8	27.0
temporal	130	71.5	78.5	71.5	60.8	36.2
open_domain	32	53.1	62.5	43.8	6.2	3.1
overall	584	68.0	73.8	65.6	42.5	26.7

1.3 The clean comparison: single_hop, original flat retrieval vs shipped stack

Same 282 questions, same code base, same judge, only retrieval changed.

Retrieval	Accuracy	recall@10	recall@50	seconds per question
flat (original Tesseract)	64.9%	39.4	61.7	6.4
hybrid (speaker boost + embedding back-fill)	67.0%	44.7	61.7	8.2
graph (time chain, same-speaker edges, dialogue links appended)	66.7%	39.4	61.7	7.9
local_pairs (per-agent routing + pair back-fill)	73.8%	46.8	62.8	7.9

Same 111 single_hop ids from conversations 1 to 4, flat vs shipped, per judge:

Judge	Flat	Shipped	Gain	Wins / losses
Gemma lenient	66.7	75.7	+9.0	32 / 7, p < .001
Qwen lenient	53.2	62.2	+9.0	45 / 13, p < .001
Qwen strict	22.5	33.3	+10.8	38 / 11, p < .001
Substring	18.0	21.6	+3.6	18 / 5, p = .01

Where the flat system loses: when the gold text reached the final 15-memory context, accuracy was 77.8%; when it did not (124 of 282 questions), 48.4%.

2. Retrieval experiments (recall, no LLM calls)

2.1 Baseline retrievers, all 1,540 questions

Retriever	single_hop @10 / @50	multi_hop @10 / @50	open_domain @10 / @50	ALL @10 / @50
pure embedding	25.9 / 52.1	35.1 / 54.8	13.5 / 19.8	25.3 / 41.9
Tesseract flat	39.4 / 61.7	46.8 / 61.4	13.5 / 20.8	34.9 / 48.0
Tesseract + graph neighbours mixed into top-50	27.3 / 38.3	38.3 / 43.5	7.3 / 10.4	27.5 / 32.9
Tesseract + graph neighbours appended (recall@all)	39.4 / 61.7 / 64.5	46.8 / 61.4 / 62.2	13.5 / 20.8 / 21.9	34.9 / 48.0 / 49.0
Tesseract + speaker boost	44.7 / 61.7	49.2 / 61.4	13.5 / 20.8	37.2 / 48.0
Tesseract + embedding back-fill (recall@all)	39.4 / 61.7 / 65.2	46.8 / 61.4 / 65.6	13.5 / 20.8 / 22.9	34.9 / 48.0 / 65.5 (3 cats)

Budget-matched conclusion: 30 extra candidates from graph neighbours add +1.4 recall@all; 30 from plain embedding add +4.1.

2.2 Pair nodes (H3): index each reply together with the message before it

Variant	single_hop @10 / @50 / @all	multi_hop @10 / @50 / @all	open_domain	ALL @10 / @50 / @all
pure embedding	25.9 / 52.1 / 52.1	35.1 / 54.8 / 54.8	13.5 / 19.8 / 19.8	31.3 / 51.4 / 51.4
embedding + pairs	41.1 / 59.6 / 59.6	54.1 / 65.2 / 65.2	12.5 / 19.8 / 19.8	47.8 / 60.3 / 60.3
Tesseract + embedding back-fill (control)	39.4 / 61.7 / 65.2	46.8 / 61.4 / 65.6	13.5 / 20.8 / 22.9	42.5 / 58.2 / 62.2

Variant	single_hop @10 / @50 / @all	multi_hop @10 / @50 / @all	open_domain	ALL @10 / @50 / @all
Tesseract + pair back-fill	39.4 / 61.7 / 68.4	46.8 / 61.4 / 69.1	13.5 / 20.8 / 24.0	42.5 / 58.2 / 65.4
window of 3 messages	39.4 / 61.7 / 68.1	46.8 / 61.4 / 69.6	13.5 / 20.8 / 20.8	42.5 / 58.2 / 65.4

Verdict: works. +3.2 recall@all over the control at the same 30-candidate budget. Window of 3 adds nothing. Extra cost: 5,872 embeddings once.

2.3 Scoring-formula fixes (H4, H6)

Configuration	single_hop @10 / @50	multi_hop @10 / @50	open_domain @10 / @50
Tesseract baseline	39.4 / 61.7	46.8 / 61.4	13.5 / 20.8
H4 keyword denominator fix (morphology + thesaurus)	39.0 / 60.3	47.1 / 60.0	13.5 / 21.9
H4 morphology only	39.0 / 61.3	47.9 / 60.6	11.5 / 20.8
H6a soft temporal penalties	39.4 / 61.7	46.8 / 61.5	13.5 / 20.8
H6b no minimum-charge cutoff	39.4 / 61.3	46.8 / 61.2	12.5 / 20.8
H6c embedding rescue	36.5 / 61.0	48.2 / 62.8	14.6 / 20.8
H4 + H6 + rescue	36.5 / 59.9	47.9 / 62.0	13.5 / 20.8

Verdict: null. The four scoring stores overlap so heavily that per-store changes reorder nothing. Surprise: the temporal store dominates the fused top-10 for every category because fusion weights are near-uniform.

2.4 Two-agent memory split (H9a): treat each speaker as an agent with its own store

Variant	single_hop @10 / @50	multi_hop @10 / @50	open_domain @10 / @50
Tesseract, pooled (control)	39.4 / 61.7	46.8 / 61.4	13.5 / 20.8
Tesseract, routed to the named agent's store	46.8 / 62.8	50.3 / 63.3	15.6 / 19.8
Tesseract, oracle routing (gold speaker)	46.5 / 61.7	51.0 / 64.0	15.6 / 19.8
Tesseract, federated by charge	36.9 / 60.3	42.4 / 59.9	11.5 / 20.8
embedding, pooled	25.9 / 52.1	35.1 / 54.8	13.5 / 19.8
embedding, routed to the named agent	46.8 / 63.8	52.8 / 64.4	13.5 / 17.7

The gold-bearing message is spoken by the other agent only 2.9% of the time (0% for single_hop), so name routing equals oracle routing. Federating by charge is worse than pooling because per-store charges are not comparable.

2.5 Speaker-swap probe (H9b): rewrite the question with the other speaker's name, 244 single_hop questions

Retriever	Original recall@10	Swapped recall@10	Gap (identity sensitivity)	Still retrieves original gold in top 10 after swap
pure embedding	26.6	34.8	-8.2	78.5%
Tesseract	42.6	32.0	+10.7	57.7%

Retriever	Original recall@10	Swapped recall@10	Gap (identity sensitivity)	Still retrieves original gold in top 10 after swap
Tesseract + speaker boost	48.8	26.2	+22.5	45.4%

Swapping the name raises pure-embedding recall. Measured cause: a speaker's own name appears in 0.1% of their messages, the other speaker's name in 33.4%. A name in the question is an embedding-level pointer to the wrong speaker.

2.6 Combined stacks with local routing (H9 follow-up), 1,219 questions (three categories)

Variant	single_hop @10 / @50 / @all	multi_hop @10 / @50 / @all	open_domain @10 / @50 / @all
embedding, local	46.8 / 63.8 / 63.8	52.8 / 64.4 / 64.4	13.5 / 17.7 / 17.7
embedding + pairs, local	41.1 / 59.2 / 59.2	53.7 / 64.0 / 64.0	14.6 / 20.8 / 20.8
Tesseract, local	46.8 / 62.8 / 62.8	50.3 / 63.3 / 63.3	15.6 / 19.8 / 19.8
Tesseract local + pair back-fill (shipped)	46.8 / 62.8 / 69.5	50.3 / 63.3 / 68.4	15.6 / 19.8 / 24.0
Tesseract local + plain embedding back-fill (control)	46.8 / 62.8 / 66.0	50.3 / 63.3 / 65.4	15.6 / 19.8 / 19.8
embedding local, top 80 (control)	46.8 / 63.8 / 66.7	52.8 / 64.4 / 66.0	13.5 / 17.7 / 21.9

Pair nodes are a good back-fill and a bad base: inside the top-50 they cost single_hop 5.7 points at recall@10 because expanding a pair pulls in the other agent's message.

2.7 Reply-only pair mapping (N3)

Variant	single_hop @10 / @50 / @all	multi_hop @10 / @50 / @all	open_domain @all
shipped (both members)	46.8 / 62.8 / 69.5	50.3 / 63.3 / 68.4	24.0
reply-only back-fill	46.8 / 62.8 / 68.4	50.3 / 63.3 / 67.1	21.9
embedding + pairs reply-only, as base	45.7 / 62.1 / 62.1	55.4 / 64.2 / 64.2	18.8
pairs only, reply-only	45.4 / 62.1 / 62.1	55.4 / 64.2 / 64.2	18.8

Verdict: negative. Of 23 questions lost, 14 had the gold only in the other agent's message; under local routing the both-member pair link is the only channel that lets it in.

2.8 Mega search (M1): vector + BM25 + entity-graph PageRank hop, fused by reciprocal rank, 1,219 questions

Retriever	single_hop @10 / @50 / @all	multi_hop @10 / @50 / @all	open_domain @10 / @50 / @all	ALL @10 / @50 / @all
shipped (Tesseract local + pairs)	46.8 / 62.8 / 69.5	50.3 / 63.3 / 68.4	15.6 / 19.8 / 24.0	46.8 / 59.7 / 65.1
vector + BM25	46.8 / 62.1 / 65.2	55.2 / 66.0 / 67.4	12.5 / 18.8 / 19.8	49.9 / 61.4 / 63.2
vector + graph hop	47.2 / 64.9 / 67.4	57.8 / 65.3 / 66.8	14.6 / 20.8 / 22.9	51.9 / 61.7 / 63.5
vector + BM25 + graph	47.5 / 62.4 / 67.0	55.2 / 64.6 / 67.3	12.5 / 21.9 / 22.9	50.0 / 60.7 / 63.7

Retriever	single_hop @10 / @50 / @all	multi_hop @10 / @50 / @all	open_domain @10 / @50 / @all	ALL @10 / @50 / @all
vector + graph + Tesseract, then pairs (regex entities)	51.1 / 65.6 / 68.1	59.1 / 66.6 / 68.5	14.6 / 21.9 / 24.0	53.7 / 62.8 / 64.9
same, LLM-built entity graph (M2)	49.6 / 64.5 / 67.7	56.7 / 66.7 / 68.6	14.6 / 19.8 / 24.0	51.8 / 62.5 / 64.9
same, LLM + regex entities	50.7 / 64.9 / 68.4	58.9 / 66.7 / 68.5	14.6 / 20.8 / 24.0	53.5 / 62.7 / 65.0
LLM graph channel alone	18.1 / 25.9 / 29.4	24.9 / 33.2 / 36.0	5.2 / 8.3 / 8.3	21.7 / 29.5 / 32.3

BM25 adds nothing once vectors and pairs exist. The graph hop is the only graph mechanism that beat plain embedding at the retrieval level (+6.9 recall@10 overall, +8.8 on multi_hop). See section 3.7 for why it still loses end to end.

3. End-to-end experiments (accuracy, same 100 single_hop ids unless noted, shipped retrieval, Gemma lenient judge)

Control for every row: 78 of 100. Gold text present in the final 15-memory context: 71%. Accuracy given gold present: 88.7%.

3.1 Listwise reranker (H1)

Reranker	Accuracy	Rerank ms	End-to- end ms	Gold in context	Accuracy given gold	LLM calls per question
pointwise 0 to 3, one call per candidate (control)	78	7,391	7,948	71%	88.7%	80
listwise, 8 batches of 10, 0 to 10	74	3,258	3,816	71%	84.5%	8
listwise + second global pass	72	5,845	6,387	71%	83.1%	9

Verdict: latency win, accuracy loss. Gold-in-context is identical across all three: the reranker only reorders, retrieval decides.

3.2 Answer context size (H2, N1)

Memories in prompt (plain / list questions)	Accuracy	Gold in context	Accuracy given gold	Answer ms
5 / 10	69	57%	87.7%	180
10 / 10	71	60%	88.3%	190
10 / 15	71	65%	83.1%	250
15 / 30 (default)	78	71%	88.7%	479
30 / 30	76	71%	87.3%	506
50 / 50	69	73%	78.1%	593

Post-rerank rank of the gold message: 1 to 5 for 52 questions, 6 to 15 for 15, 16 to 30 for 4, absent for 29. Widening adds no gold and distracts the model; narrowing loses corroborating second mentions. The default is the optimum.

3.3 LLM router vs regex router (H7)

Category	Regex (control)	LLM router	Agreement	Added latency
single_hop	78 / 100	79 / 100	52%	+105 ms
temporal	78 / 100	76 / 100	97%	+119 ms

Verdict: null. Regex already sends 97 of 100 temporal questions to the temporal mode. The two routers disagree on half of single_hop yet score the same.

3.4 Open-domain prompt flags (H5), all 96 open_domain questions

Configuration	Accuracy	Abstentions
control	41 / 96 = 42.7%	27
keep context in world mode + hedge routing	43 / 96 = 44.8%	
+ memories-plus-world-knowledge prompt for inference mode	55 / 96 = 57.3%	13

single_hop with the flags on: 77 vs 78 (no regression). Under the strict independent judge the gain disappears (see 4.2); about 4 of the 17 wins are judge leniency on hedged answers.

3.5 Pair-aware reranker and answer prompt (N4)

Configuration	Accuracy	Rerank ms	Gold in context	Accuracy given gold
control	78	7,349	71%	88.7%
reranker sees previous message	71	8,479	69%	82.6%
answer prompt shows previous message	75	7,989	73%	83.6%
both	66	8,197	70%	78.6%

Verdict: negative. The previous line dilutes the candidate rather than explaining it. Pair context belongs in retrieval only.

3.6 Answer model swap (H8): Gemma-4-e4b vs Qwen3.6-35b-a3b for the answer call only

	Gemma answers	Qwen answers
accuracy, Gemma lenient judge	78	71
accuracy, Qwen lenient judge	64	65
accuracy, Qwen strict judge	38	42
substring	24	29
gold in context	71	71
"Not found" answers with gold present	4	11
answer latency ms	501	913

Verdict: a judge artifact. The sign flips once the judge is not the answering model's family.

3.7 Mega search and LLM-built graph end to end (M1, M2)

Category, same 100 ids	Shipped	Regex-entity mega search	LLM-entity mega search
single_hop	78	68 (3 wins / 13 losses)	70 (4 / 12)
multi_hop	75	76	77 (8 / 6)

Accuracy given gold in context on single_hop: 88.6 shipped, 79.1 regex graph, 82.6 LLM graph. The entity hop fills the candidate window with topically related non-answers that the reranker and answer model prefer. LLM extraction (138 to 354 resolved entities and 430 to 712 triples per conversation, versus about 2,000 regex entities) recovers 2 points and does not fix it.

4. Judge sensitivity (N5)

4.1 Same answer files, four graders

Answer file	n	Gemma lenient (campaign)	Qwen lenient	Qwen strict	Substring
flat, single_hop	282	64.9	51.4	21.6	13.5
shipped, single_hop	282	73.8	62.8	31.2	18.1
H8 Gemma answers	100	78.0	64.0	38.0	24.0
H8 Qwen answers	100	71.0	65.0	42.0	29.0
H5 control, open_domain	96	42.7	27.1	11.5	6.2
H5 treatment, open_domain	96	57.3	39.6	10.4	9.4

4.2 Does the verdict survive the judge?

Pair	Gemma lenient	Qwen lenient	Qwen strict	Substring	Verdict
flat to shipped	+8.9	+11.3	+9.6	+4.6	stable, works under all
Gemma to Qwen answers (H8)	-7.0	+1.0	+4.0	+5.0	sign flips, null
open-domain flags (H5)	+14.6	+12.5	-1.0	+3.1	vanishes under strict, null

Agreement with the campaign judge: Qwen lenient 86.5% (kappa 0.72), Qwen strict 59.4% (kappa 0.30), substring 49.1% (kappa 0.17). Gemma-correct answers average 53 characters, wrong ones 21; the strict judge shows no length bias. Judge model alone moves scores about 12 points with an identical prompt; prompt leniency about 30 points, almost all on multi-item gold answers (66% of single_hop golds).

5. Summary table of every experiment

#	Experiment	Level	Result	Verdict
0	Leakage removal	harness	scores now honest	required
H9	Per-agent memory routing	retrieval + e2e	recall@10 +7.4; e2e +8.9 with pairs	works
H3	Pair nodes as back-fill	retrieval	+3.2 recall@all	works
H5	Open-domain prompt flags	e2e	+14.6 lenient, -1.0 strict	lenient-judge only
hybrid	Speaker boost + embedding back-fill	e2e	+2.1	small
graph	Graph-neighbour expansion	retrieval + e2e	+1.4 recall, +1.8 e2e	no better than embedding back-fill
H1	Listwise reranker	e2e	-4, latency -56%	speed only
H2	Wider context (30, 50)	e2e	-2, -9	hurts

#	Experiment	Level	Result	Verdict
N1	Narrower context (10, 5)	e2e	-7, -9	hurts
H7	LLM router	e2e	+1 / -2	null
H4	Keyword formula fix	retrieval	-0.4 to -1.4	null
H6	Softer temporal penalties	retrieval	within 0.2	null
N3	Reply-only pairs	retrieval	-1 to -2	hurts
N4	Pair-aware reranker / prompt	e2e	-7 / -3 / -12	hurts
H8	35B answer model	e2e	-7 lenient, +4 strict	judge artifact
M1	Mega search (vector + BM25 + graph hop)	retrieval + e2e	+6.9 recall@10; e2e -10 single_hop, +1 multi_hop	hurts
M2	LLM-built entity graph in mega search	retrieval + e2e	+5.0 recall@10; e2e -8 single_hop, +2 multi_hop	hurts
N5	Judge sensitivity	evaluation	51-point spread by grader	paper-grade

Files: per-question results in `demo/results/*.json`; per-experiment reports in `docs/research/`; rescoring script `docs/research/n5_rescore.py`.

6. Latency per stage (seconds per question, mean; last column p95 end to end)

Measured on the same Mac and LM Studio server (gemma-4-e4b) unless marked old. Rerank = one model call per candidate (50 for flat, 80 for the shipped stack). Judge time is excluded.

Run	n	retrieve	rerank	answer	end-to-end mean	end-to-end p95
flat retrieval (original), single_hop	282	0.14	5.60	0.68	6.43	7.44
hybrid, single_hop	282	0.13	7.51	0.55	8.20	9.36
graph neighbours appended, single_hop	282	0.13	7.37	0.44	7.94	8.25
shipped local_pairs, single_hop	282	0.08	7.34	0.46	7.89	8.24
shipped, conversations 1-4, all categories	590	0.08	7.68	0.39	8.17	10.99
shipped, 100 single_hop (control)	100	0.07	7.39	0.48	7.95	8.27
Qwen-35B answer model, 100 single_hop	100	0.08	7.28	0.89	8.26	8.88
open-domain flags, 96 open_domain	96	0.09	7.39	0.32	7.83	8.24
mega search (regex graph), 100 single_hop	100	0.10	7.48	0.51	8.10	8.51
mega search (LLM graph), 100 single_hop	100	0.09	7.28	0.48	7.86	8.20
mega search (regex graph), 100 multi_hop	100	0.08	7.39	0.46	7.93	8.31
30-memory context, 100 single_hop (H2_treatment30.json)	100	0.07	7.36	0.51	7.95	8.25
old Dec-2025 run (Ollama, qwen2.5-7b, leaky)	1540	0.33	1.49	0.39	2.41	3.15

Reading: retrieval is 80 to 140 ms and gets faster with per-agent stores; the shipped stack costs about 2 s more per question than flat because it reranks 80 candidates instead of 50, which is the price of the +9 accuracy. The listwise reranker halves rerank time at a 4-point accuracy cost. Old-run rerank latency reflects a different server (Ollama) and model, not the architecture.